Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 01

Desarrollo de aplicaciones Web: Configuración del Entorno de Desarrollo

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: CONFIGURACIÓN DEL ENTORNO DE DESARROLLO

I. Capacidades

- Instalación y configuración de IDE.
- Creación de proyecto inicial.
- Ejecución de aplicación básica.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

VII. Procedimiento

CONFIGURACIÓN DE ENTORNO DE TRABAJO

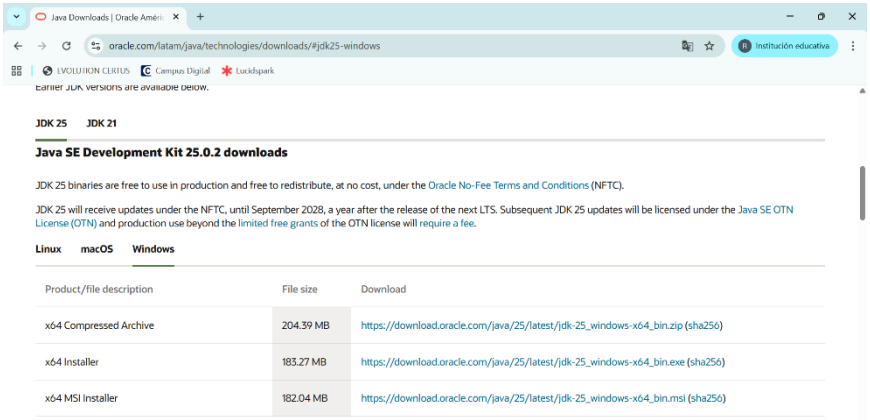
INSTALACIÓN DE LA BASE DE JAVA JDK

1. Proceso de instalación de Java JDK

1.1. Ingrese al enlace para descargar JDK (OJO: es posible que le pida crear una cuenta para permitir la descarga)

https://www.oracle.com/pe/java/technologies/downloads/#jdk22-windows

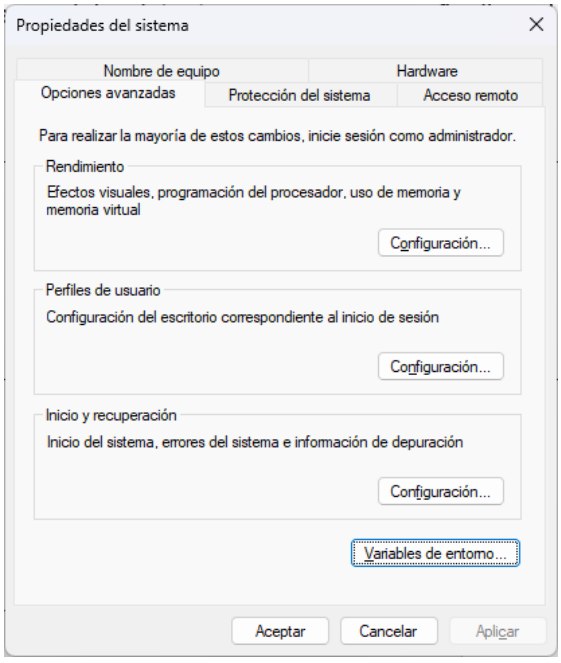
1.2. En la sección JDK Development Kit 25.0.2 elija el instalador dependiendo del sistema operativo que esté utilizando.



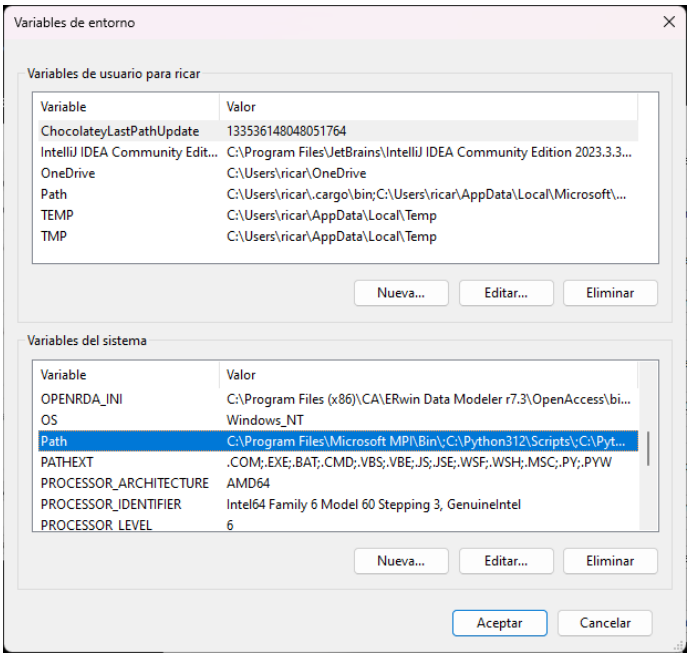
1.3. Ejecute el instalador, siga los pasos del asistente y espere a que finalice el proceso de instalación.

2. Variable de Entorno

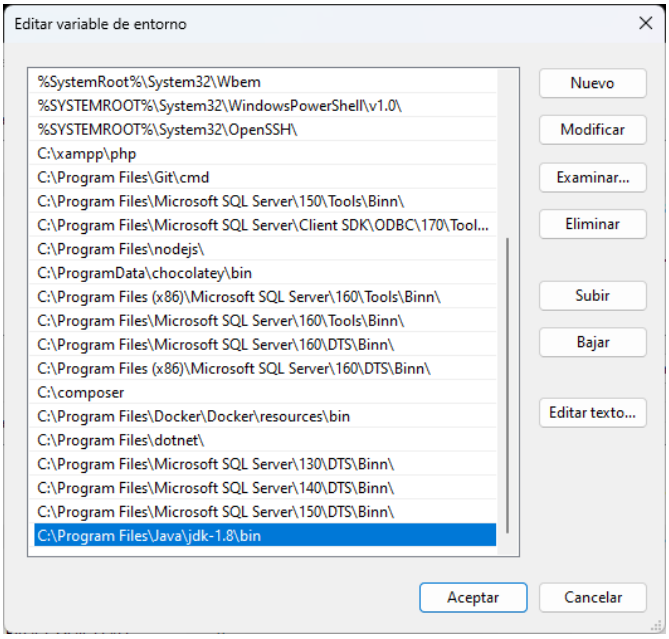
2.1. Ingresar a la siguiente ruta y copiar C:\Program Files\Java\jdk-25.0.2\bin
2.2. Presione tecla Windows / variables de entorno
2.3. Seleccione Variables de entorno…



2.4. Seleccione Path / Editar…



2.5. Seleccione Nuevo / Aceptar

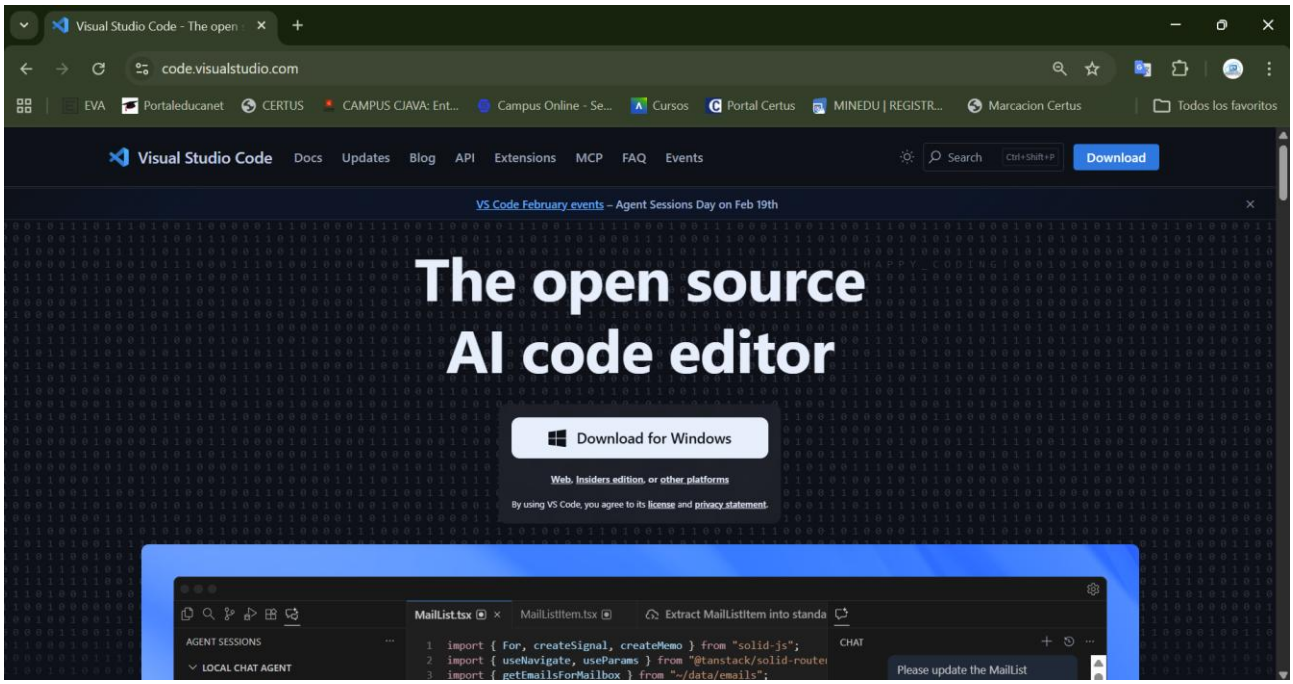


2.6. Desde una ventana de comandos ejecute la instrucción: **c:> java –version**

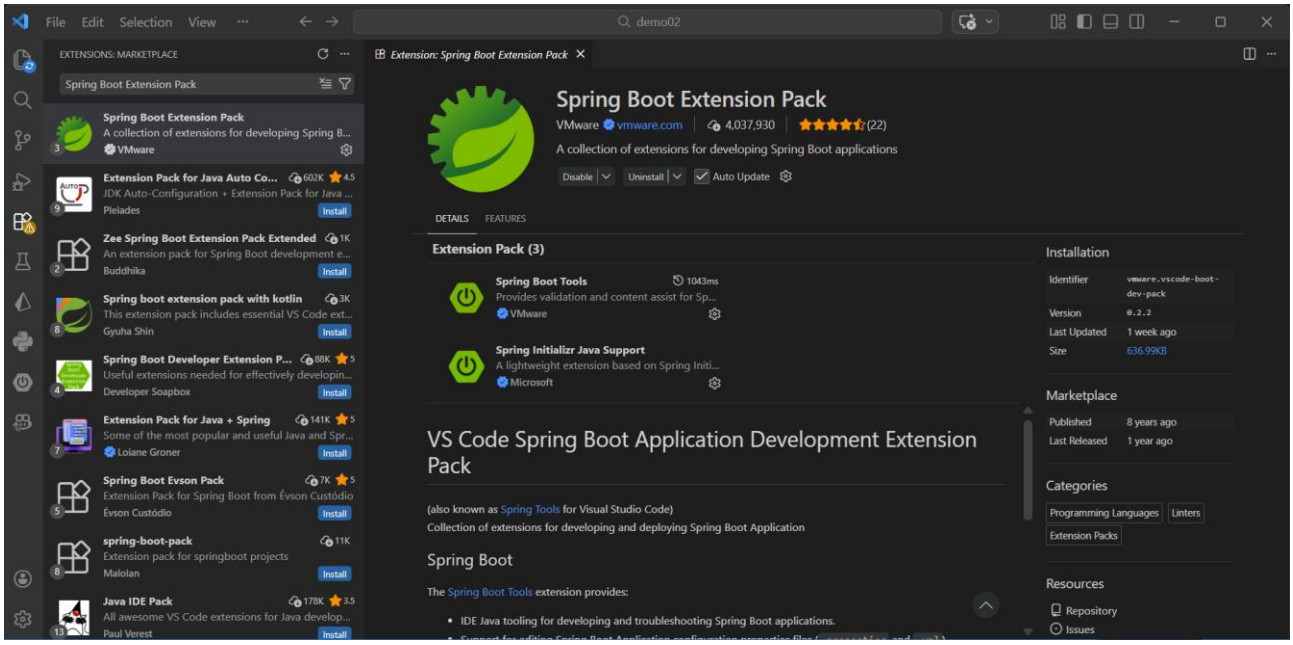
IDE

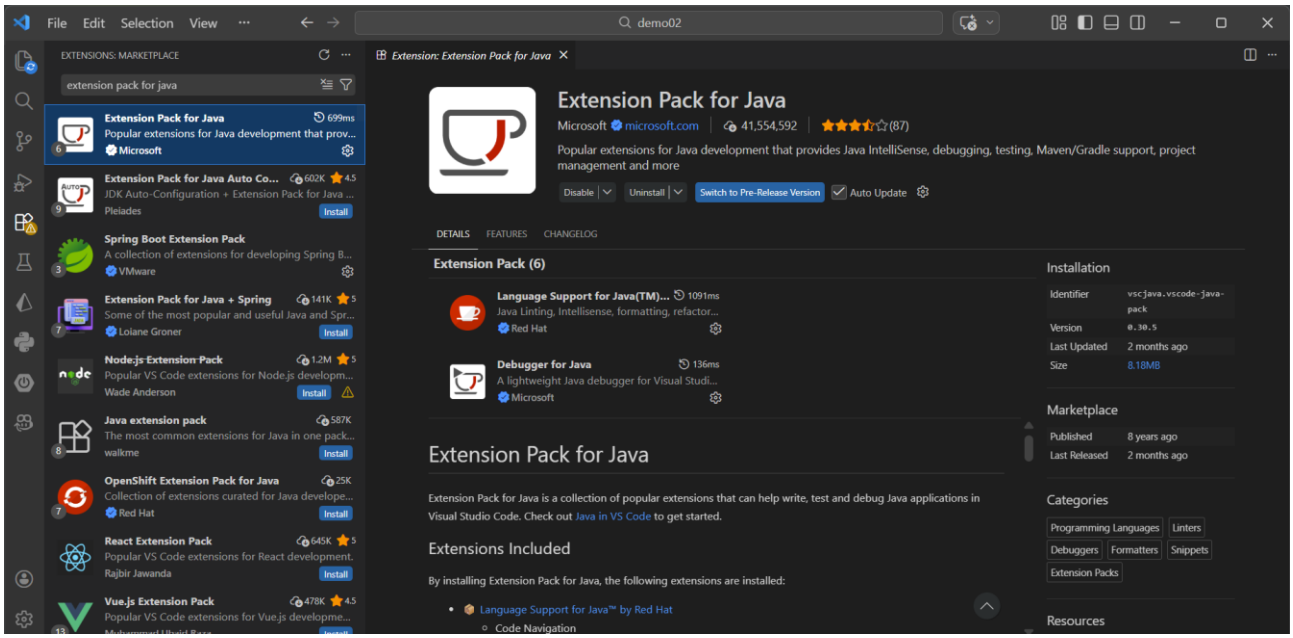
1.- Descargar e instalar Visual Studio Code

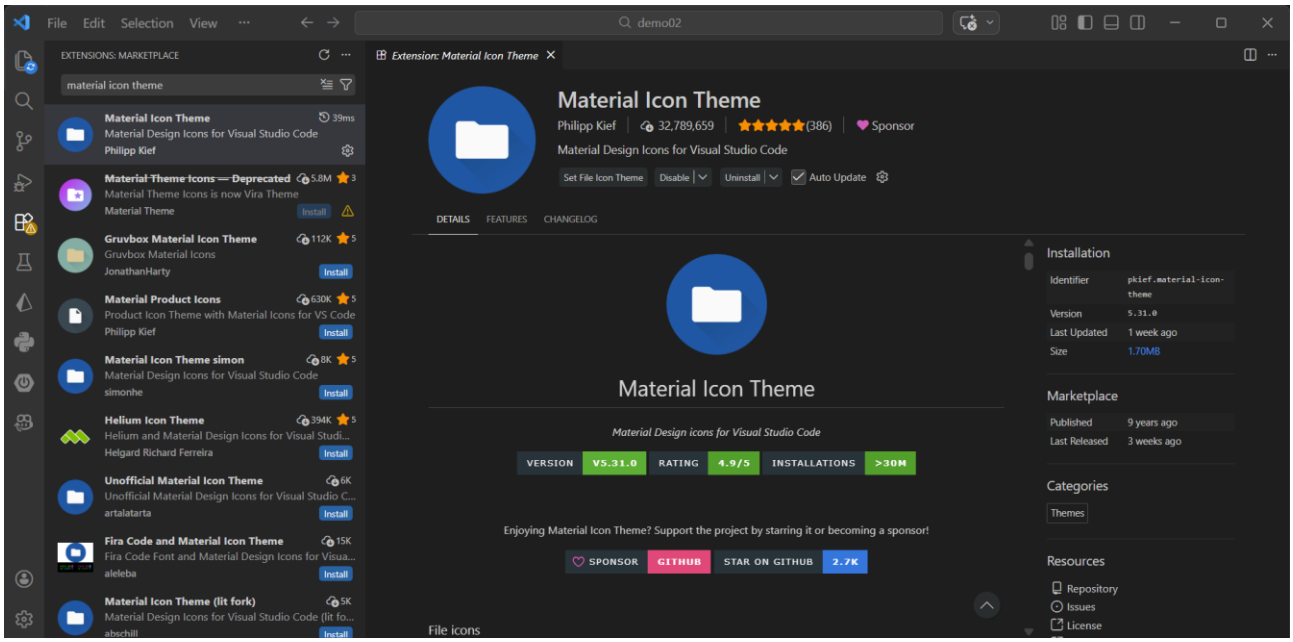
https://code.visualstudio.com/



2.- Instalar la siguiente extensión.

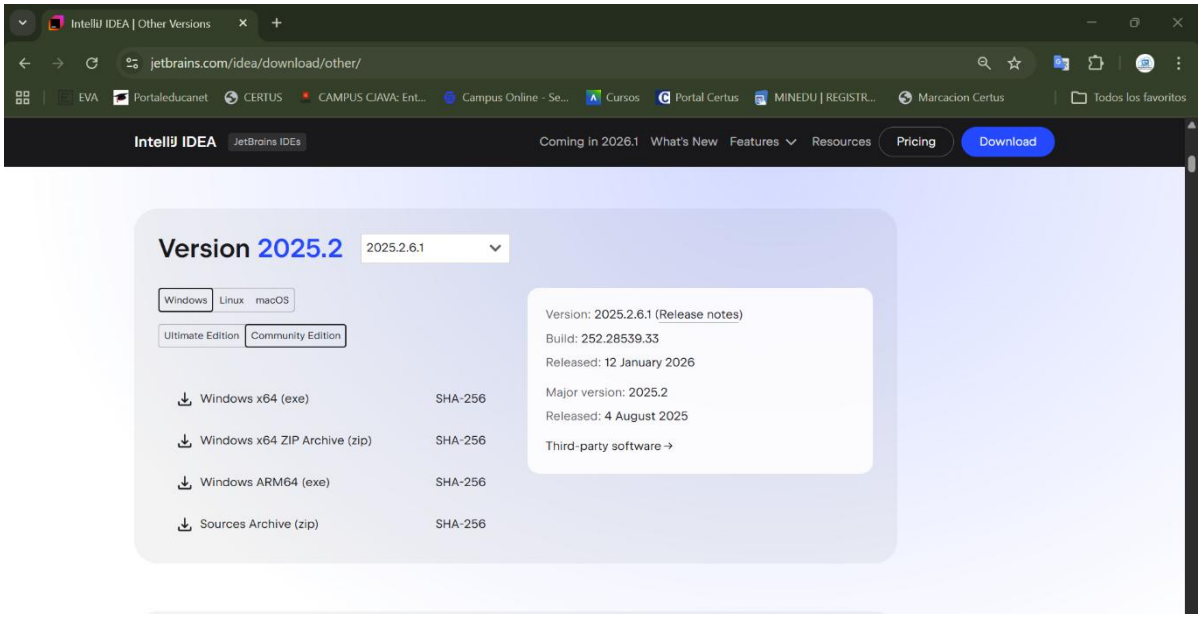






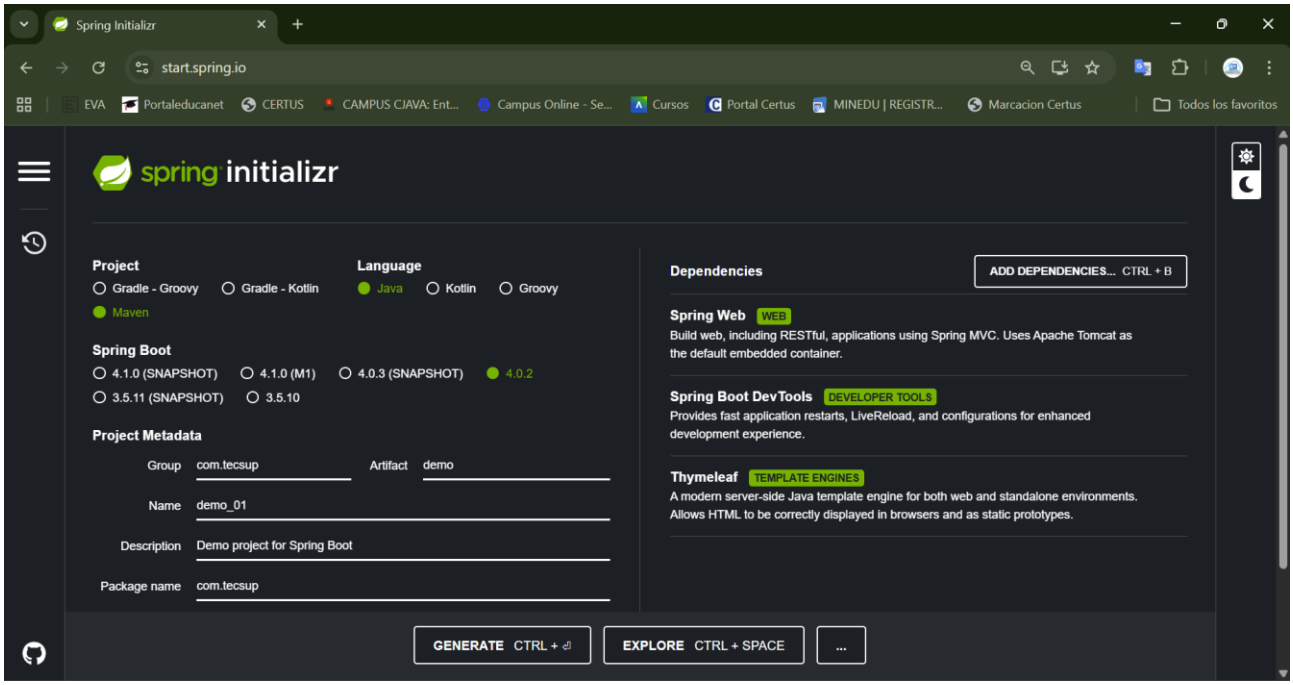
Intellij Comunity

https://www.jetbrains.com/idea/download/other/

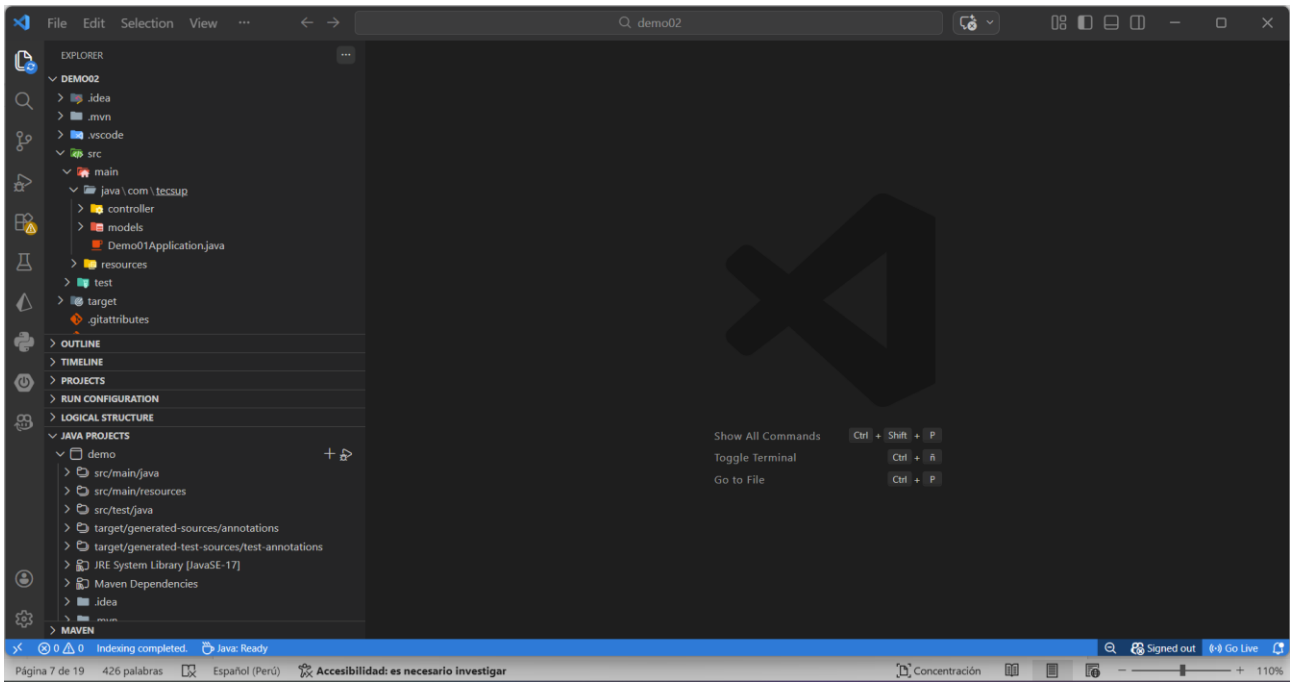


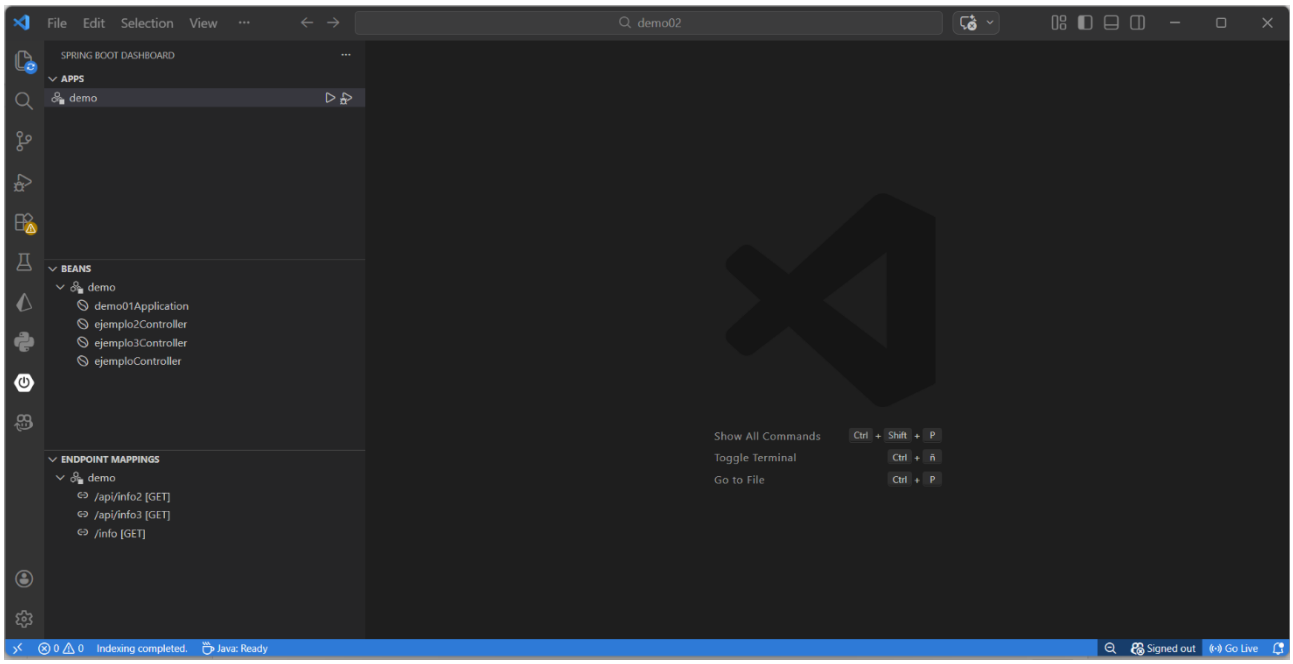
CREANDO PROYECTO CON SPRING INITIALIZR

1.



2.- Descargar el archivo y abrir en Visual Studio Code.





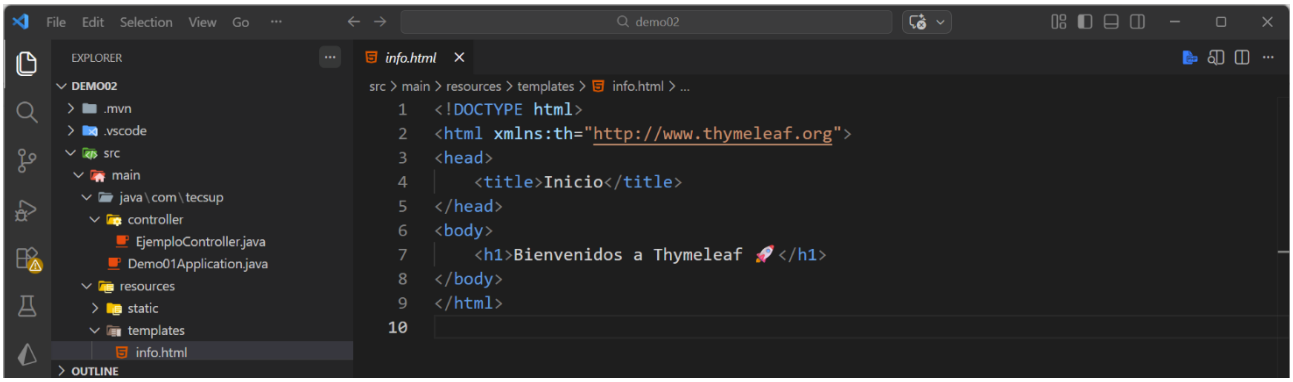
Controlador

1.- Crear la carpeta controller y la clase EjemploController

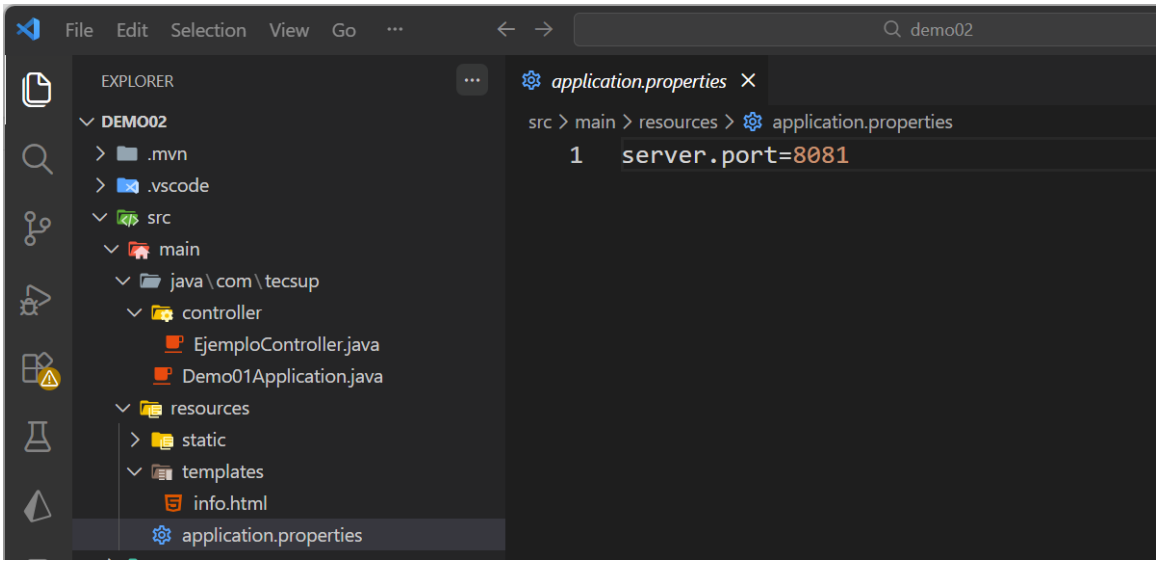


Anotación	¿Qué devuelve?	¿Busca HTML?
@RestController	Texto / JSON	❌ No
@Controller	Vista (HTML) Thymeleaf	✅ Sí

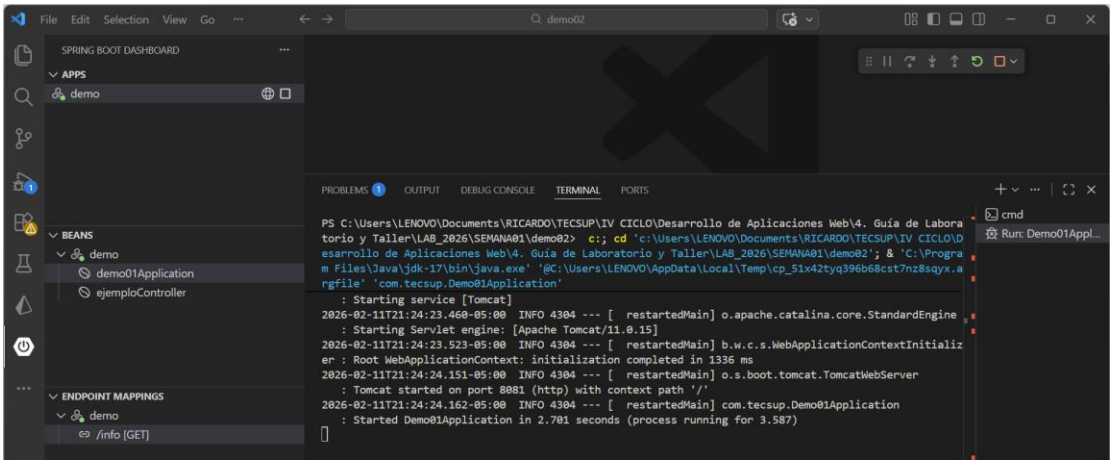
2.- Crear el archivo HTML “info”



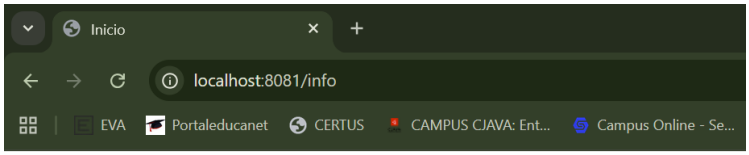
3.- Cambiar el puerto.



4.- Seleccionar el icono “Spring Boot Dashboard”, luego ejecute **demo** del **APPS**



5.- Ingrese el siguiente link.



Bienvenidos a Thymeleaf 🚀

PASAR DATOS A LA VISTA

1.- Realizar cambios en el controlador.

EjemploController.java

src > main > java > com > tecsup > controller > EjemploController.java > ...

```java
package com.tecsup.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class EjemploController {

    @GetMapping("/info")

    public String info(Model model){

        model.addAttribute("Nombre","Ricardo");
        model.addAttribute("Apellidos","Coello Palomino");

        return "info";
    }
}

```

2.- Realizar cambios en la vista.

info.html

src > main > resources > templates > info.html > ...

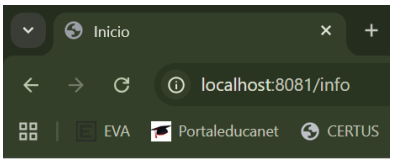
```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Inicio</title>
</head>
<body>
    <h1>Datos</h1>

    <p th:text="${Nombre}"></p>
    <p th:text="${Apellidos}"></p>

</body>
</html>

```

3.- Ejecutar



Datos

Ricardo

Coello Palomino

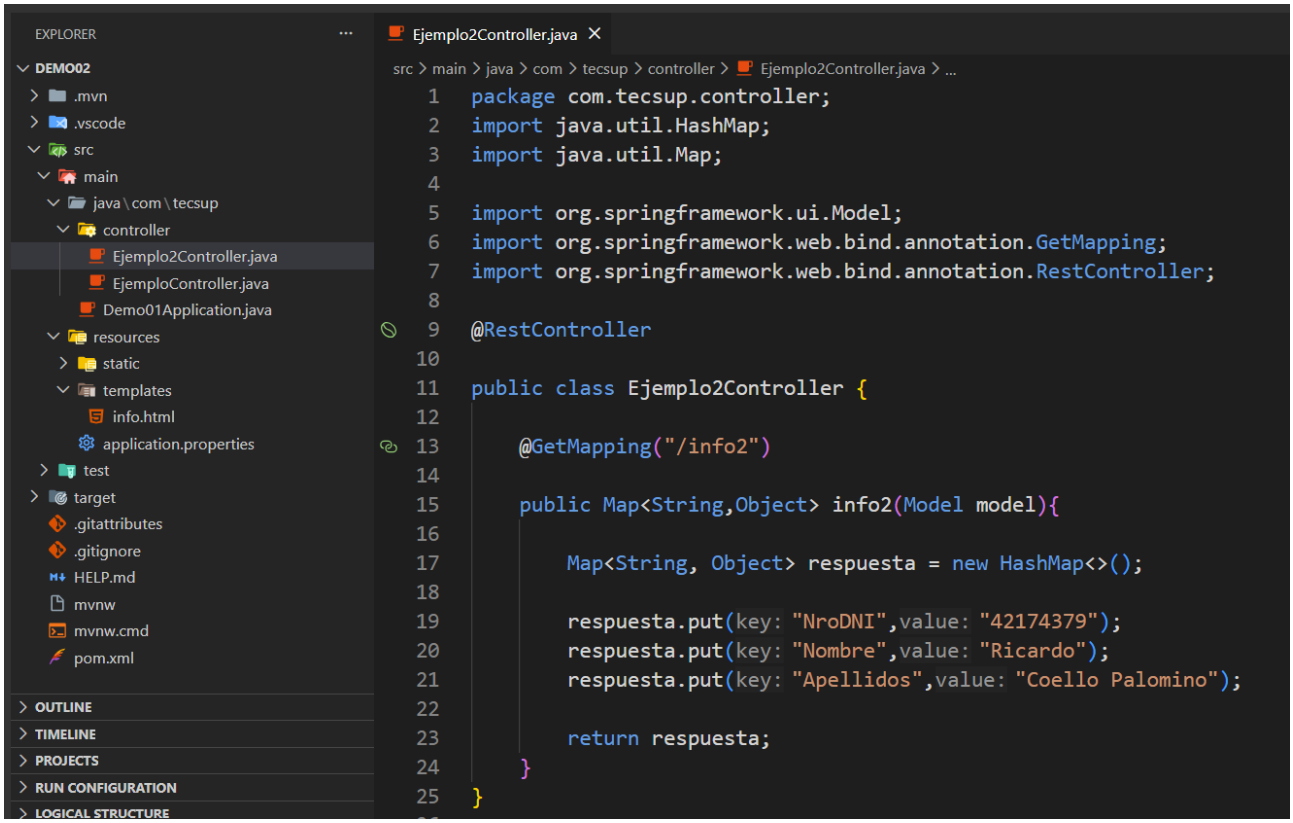
Nota: Puedes emplear Map

```java
public String info(Map<String, Object> modelo){
    modelo.put(key:"Titulo", value:"Servidor en linea");
    modelo.put(key:"Servidor", value:"Informaticonfig");
    modelo.put(key:"Ip", value:"192.168.1.1");
    return "detalles_info";

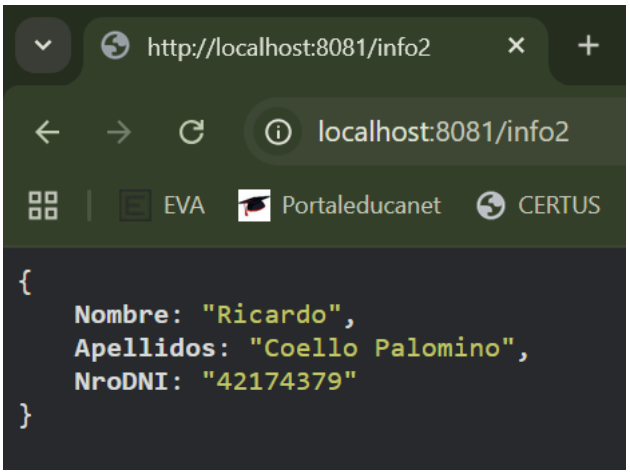
}
```

Usando RestController

1.- Crear la clase Ejemplo2Controller



2.- Ejecutar



RequestMapping

En el siguiente ejemplo agregamos el Requestmapping cuál sería la ruta de primer nivel
1.- Realizar cambios en el controlador

Ejemplo2Controller.java

src > main > java > com > tecsup > controller > Ejemplo2Controller.java > ...

```java
package com.tecsup.controller;
import java.util.HashMap;
import java.util.Map;

import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class Ejemplo2Controller {

    @GetMapping("/info2")

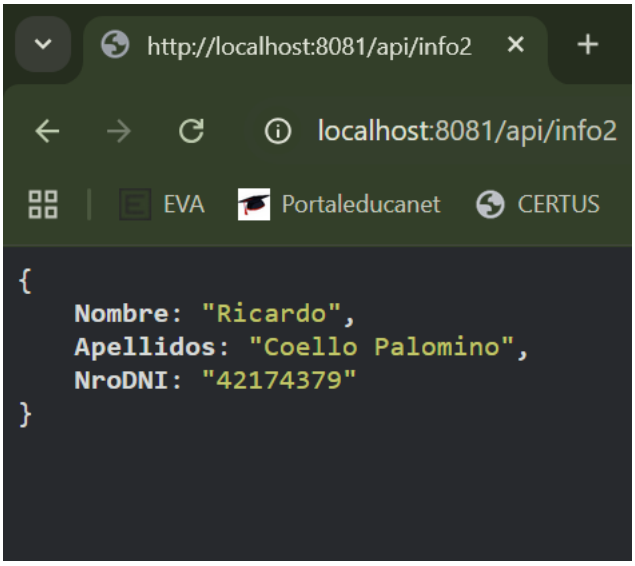
    public Map<String,Object> info2(Model model){

        Map<String, Object> respuesta = new HashMap<>();

        respuesta.put(key: "NroDNI",value: "42174379");
        respuesta.put(key: "Nombre",value: "Ricardo");
        respuesta.put(key: "Apellidos",value: "Coello Palomino");

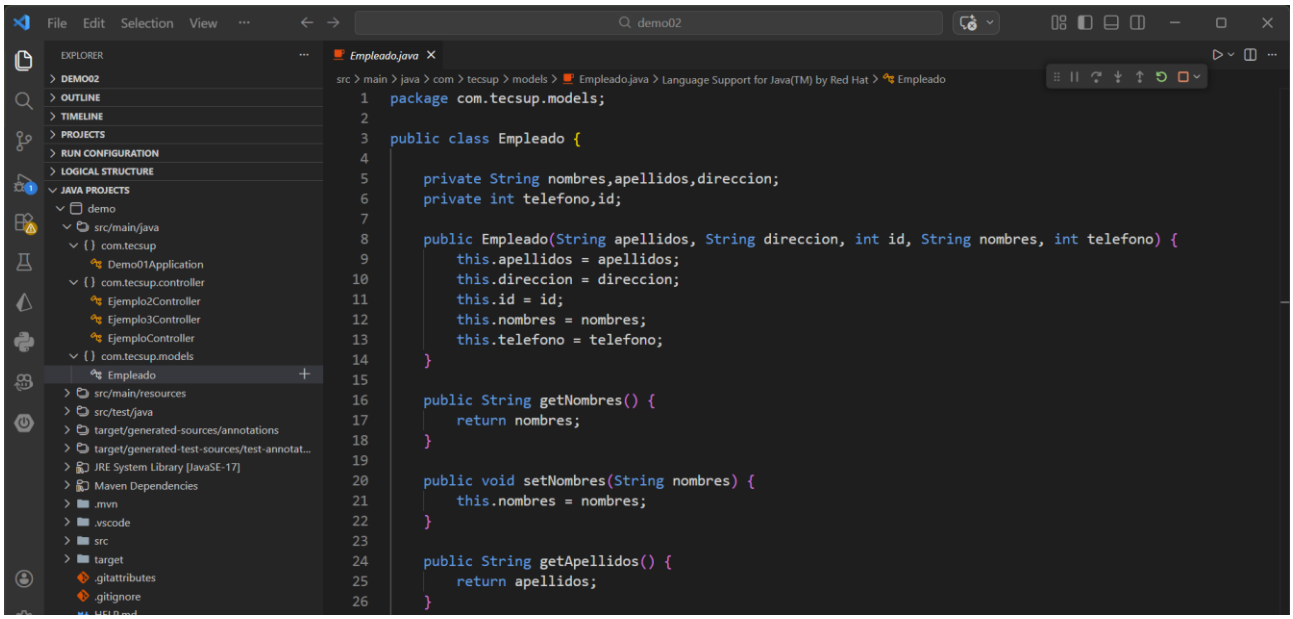
        return respuesta;
    }
}
```

2.- Ejecutar



OBJETO MODEL

1.- Crear la carpeta models y la clase Empleado.



```java
package com.tecsup.models;

public class Empleado {

    private String nombres,apellidos,direccion;
    private int telefono,id;

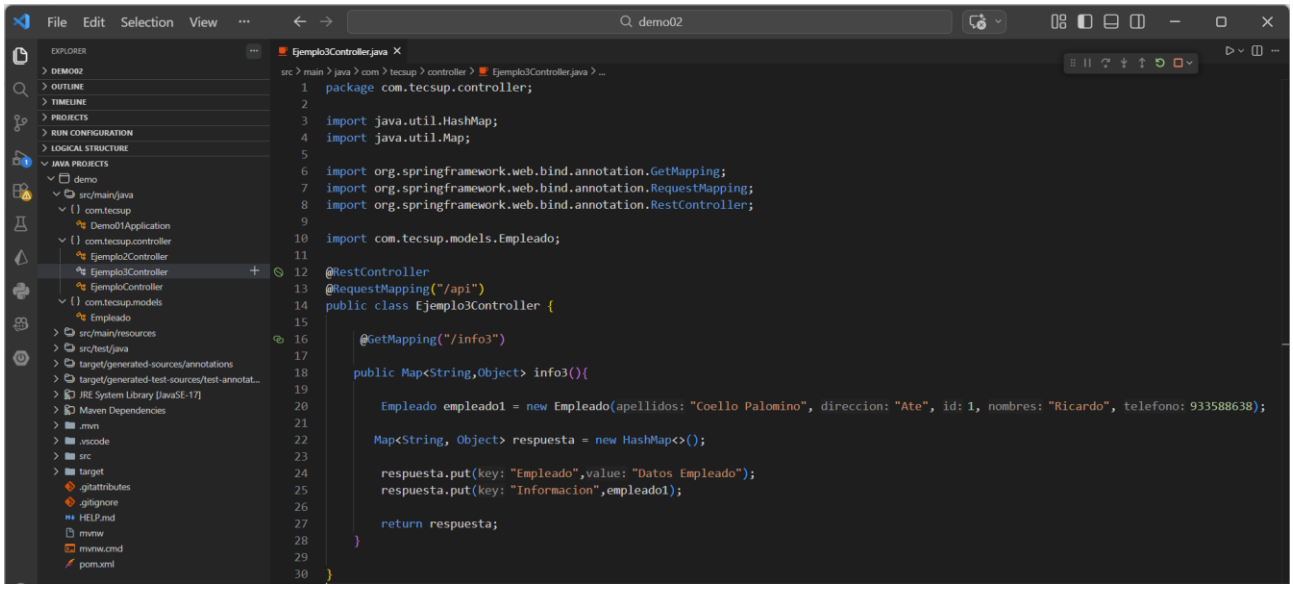
    public Empleado(String apellidos, String direccion, int id, String nombres, int telefono) {
        this.apellidos = apellidos;
        this.direccion = direccion;
        this.id = id;
        this.nombres = nombres;
        this.telefono = telefono;
    }

    public String getNombres() {
        return nombres;
    }

    public void setNombres(String nombres) {
        this.nombres = nombres;
    }

    public String getApellidos() {
        return apellidos;
    }
```

2.- Crear controlador



```java
package com.tecsup.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.tecsup.models.Empleado;

@RestController
@RequestMapping("/api")
public class Ejemplo3Controller {

    @GetMapping("/info3")

    public Map<String,Object> info3(){

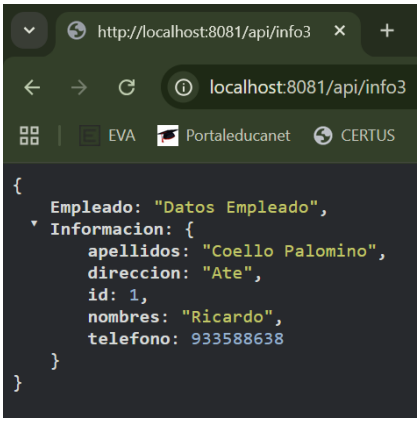
        Empleado empleado1 = new Empleado(apellidos: "Coello Palomino", direccion: "Ate", id: 1, nombres: "Ricardo", telefono: 933588638);

       Map<String, Object> respuesta = new HashMap<>();

        respuesta.put(key: "Empleado",value: "Datos Empleado");
        respuesta.put(key: "Informacion",empleado1);

        return respuesta;
    }
}
```

3.- Ejecutar



```
http://localhost:8081/api/info3
localhost:8081/api/info3
EVA  Portaleducanet  CERTUS

{
   Empleado: "Datos Empleado",
 ▾ Informacion: {
       apellidos: "Coello Palomino",
       direccion: "Ate",
       id: 1,
       nombres: "Ricardo",
       telefono: 933588638
   }
}
```

Ahora codificaremos en un **@Controller**, con pagina HTML

```java
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import com.informaticonfig.spring.app1.springboot_app1.models.Empleados;


@Controller
public class EjemploController {

   @GetMapping("/detalles_info")

   public String info(Model model){
     Empleados empleado1 = new Empleados(nombre:"Juan",apellido:"Rodriguez",direccion:"Calle 1 No 2",pu
                                         edad:35,telefono:123456789,id:001);
      model.addAttribute(attributeName:"Empleado", empleado1);

        return "detalles_info";

   }

```

```html
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title th:text="${Titulo}">Pagina Spring Boot</title>
</head>
<body>
    <h1>Información de empleado</h1>
    <ul>
        <li><strong>Nombre:</strong> <span th:text="${Empleado.nombre}"></span></li>
        <li><strong>Apellido:</strong> <span th:text="${Empleado.apellido}"></span></li>
        <li><strong>Dirección:</strong> <span th:text="${Empleado.direccion}"></span></li>
        <li><strong>Puesto:</strong> <span th:text="${Empleado.puesto}"></span></li>
        <li><strong>Edad:</strong> <span th:text="${Empleado.edad}"></span></li>
        <li><strong>Teléfono:</strong> <span th:text="${Empleado.telefono}"></span></li>
        <li><strong>ID:</strong> <span th:text="${Empleado.id}"></span></li>
    </ul>


</body>
</html>
```

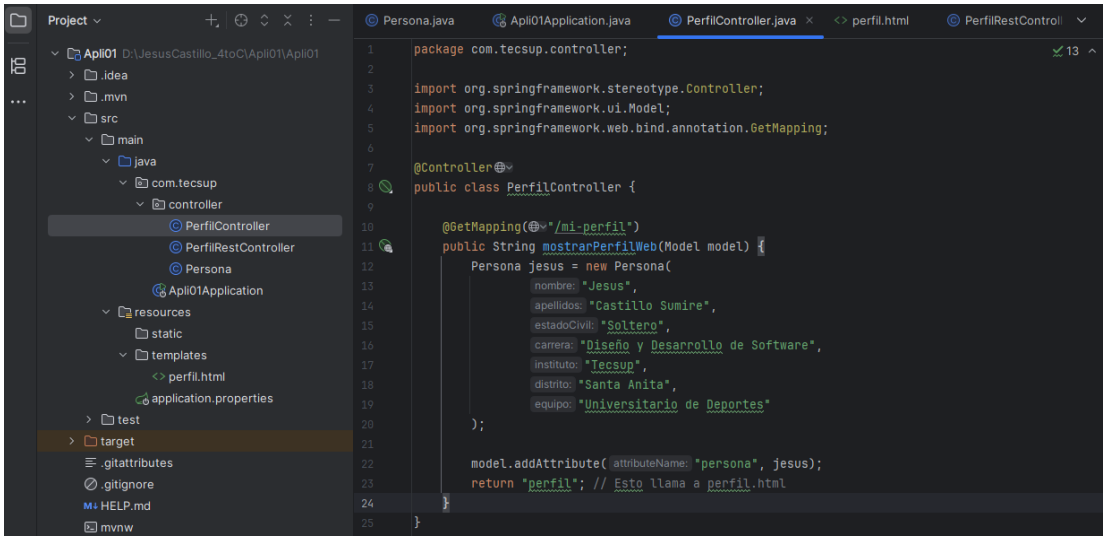
ACTIVIDAD

Aplicación Web “Información Personal”

Aplicar los conceptos aprendidos en el laboratorio:

- Controladores
- Vistas HTML
- Uso de Model
- Uso de RestController

CODIGO:



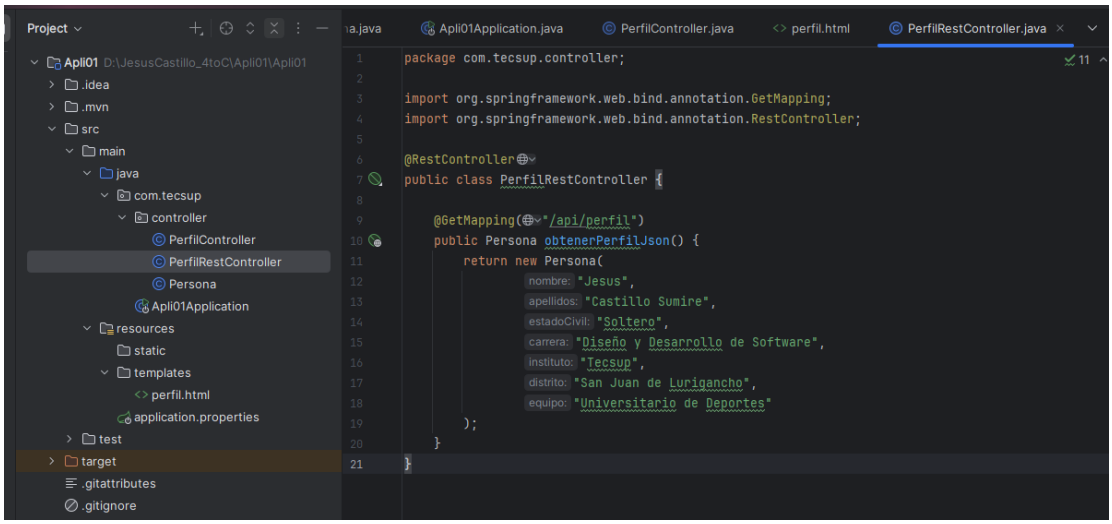
```
package com.tecsup.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class PerfilController {

    @GetMapping("/mi-perfil")
    public String mostrarPerfilWeb(Model model) {
        Persona jesus = new Persona(
                nombre: "Jesus",
                apellidos: "Castillo Sumire",
                estadoCivil: "Soltero",
                carrera: "Diseño y Desarrollo de Software",
                instituto: "Tecsup",
                distrito: "Santa Anita",
                equipo: "Universitario de Deportes"
        );

        model.addAttribute( attributeName: "persona", jesus);
        return "perfil"; // Esto llama a perfil.html
    }
}
```

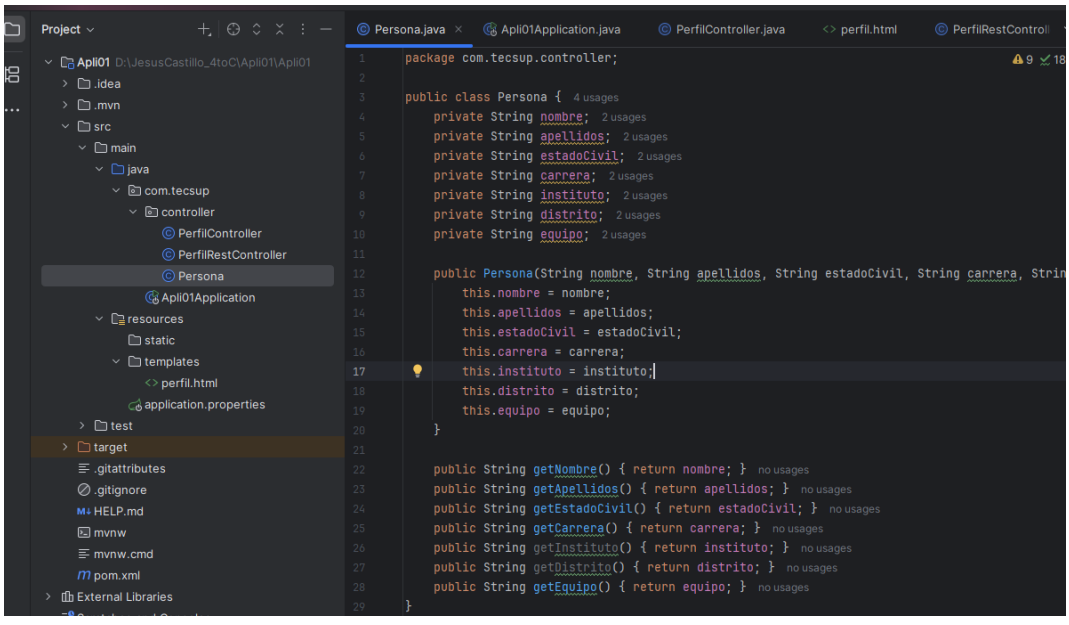


```
package com.tecsup.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class PerfilRestController {

    @GetMapping("/api/perfil")
    public Persona obtenerPerfilJson() {
        return new Persona(
                nombre: "Jesus",
                apellidos: "Castillo Sumire",
                estadoCivil: "Soltero",
                carrera: "Diseño y Desarrollo de Software",
                instituto: "Tecsup",
                distrito: "San Juan de Lurigancho",
                equipo: "Universitario de Deportes"
        );
    }
}
```

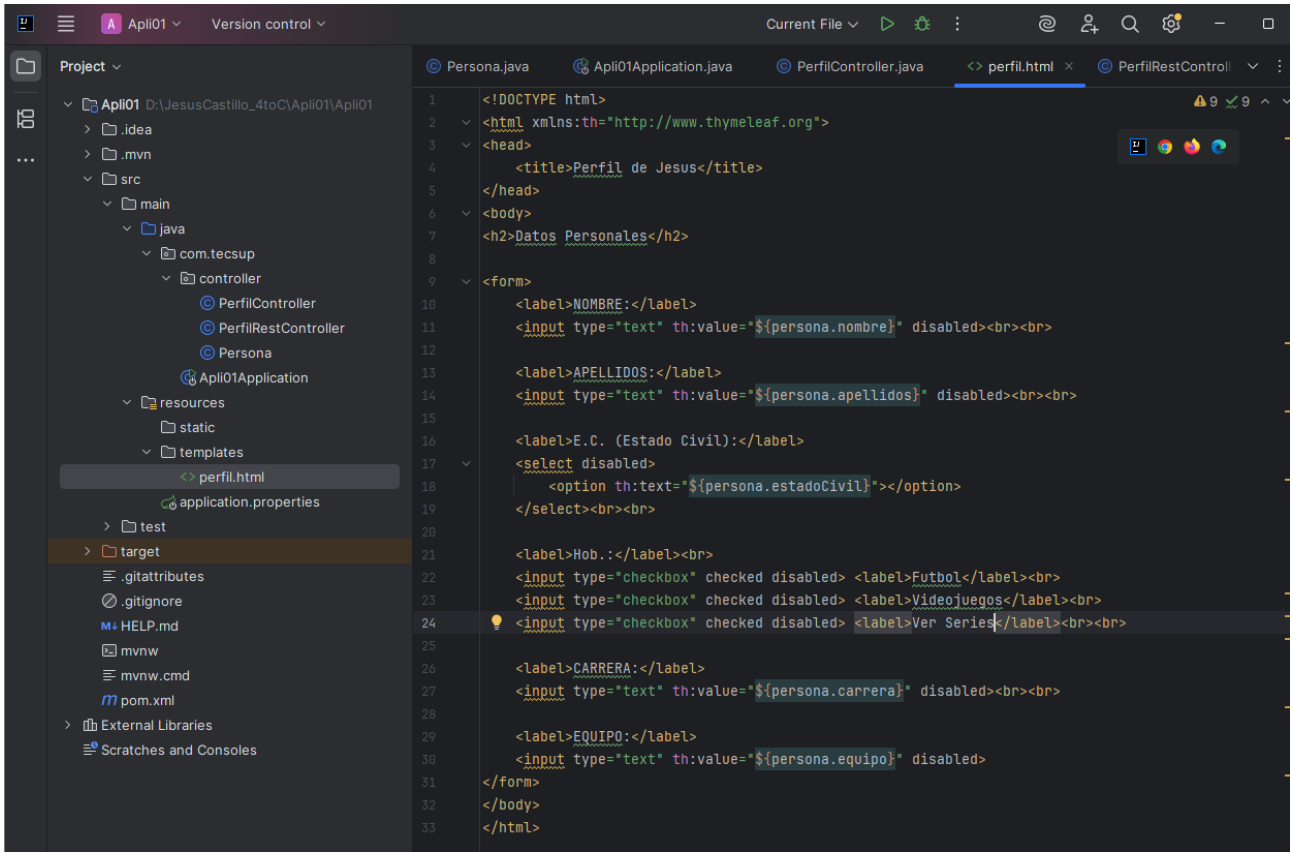


```
package com.tecsup.controller;

public class Persona {
    private String nombre;
    private String apellidos;
    private String estadoCivil;
    private String carrera;
    private String instituto;
    private String distrito;
    private String equipo;

    public Persona(String nombre, String apellidos, String estadoCivil, String carrera, Strin
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.estadoCivil = estadoCivil;
        this.carrera = carrera;
        this.instituto = instituto;
        this.distrito = distrito;
        this.equipo = equipo;
    }

    public String getNombre() { return nombre; }
    public String getApellidos() { return apellidos; }
    public String getEstadoCivil() { return estadoCivil; }
    public String getCarrera() { return carrera; }
    public String getInstituto() { return instituto; }
    public String getDistrito() { return distrito; }
    public String getEquipo() { return equipo; }
}
```


Apli01
Version control
Current File
Project
Apli01 D:\JesusCastillo_4toC\Apli01\Apli01
.idea
.mvn
src
main
java
com.tecsup
controller
PerfilController
PerfilRestController
Persona
Apli01Application
resources
static
templates
perfil.html
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
Persona.java
Apli01Application.java
PerfilController.java
perfil.html
PerfilRestControll
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<title>Perfil de Jesus</title>
</head>
<body>
<h2>Datos Personales</h2>
<form>
<label>NOMBRE:</label>
<input type="text" th:value="${persona.nombre}" disabled>

<label>APELLIDOS:</label>
<input type="text" th:value="${persona.apellidos}" disabled>

<label>E.C. (Estado Civil):</label>
<select disabled>
<option th:text="${persona.estadoCivil}"></option>
</select>

<label>Hob.:</label>
<input type="checkbox" checked disabled> <label>Futbol</label>
<input type="checkbox" checked disabled> <label>Videojuegos</label>
<input type="checkbox" checked disabled> <label>Ver Series</label>

<label>CARRERA:</label>
<input type="text" th:value="${persona.carrera}" disabled>

<label>EQUIPO:</label>
<input type="text" th:value="${persona.equipo}" disabled>
</form>
</body>
</html>

RESULTADO:


localhost:8080/mi-perfil
Datos Personales
NOMBRE: Jesus
APELLIDOS: Castillo Sumire
E.C. (Estado Civil): Soltero
Hob.:
Futbol
Videojuegos
Ver Series
CARRERA: Diseño y Desarrollo de Softw
EQUIPO: Universitario de Deportes

Preguntas de reflexión (muy buenas para evaluar)

¿Qué diferencia existe entre @Controller y @RestController?

- Por lo que he estado viendo, uso @Controller cuando necesito que mi aplicación devuelva una vista, como una página web. En cambio, @RestController lo aplico cuando armo APIs, porque ya viene con todo listo para devolver directamente los datos en formato JSON sin necesidad de agregar más anotaciones.

¿Para qué sirve el objeto Model?

- Lo uso básicamente como un puente. Me sirve para empaquetar los datos que proceso en mi controlador y pasárselos a la vista (el HTML) para que el usuario pueda verlos en la pantalla.

¿Qué función cumple Spring Initializr?

- Es una página web que me salva un montón de tiempo. La uso para generar toda la estructura inicial de mi proyecto en Spring Boot. Ahí mismo escojo la versión de Java, le meto las dependencias que voy a necesitar y me descarga el proyecto listo para abrirlo en el editor.

¿Qué carpeta contiene las páginas HTML en Spring Boot?

- Normalmente las guardo en la ruta src/main/resources/templates cuando trabajo con plantillas dinámicas. Si son archivos estáticos puros que no van a cambiar, los dejo en src/main/resources/static.

¿Qué archivo permite cambiar el puerto de la aplicación?

- Eso lo configuro rapidísimo en el archivo application.properties. Solo tengo que entrar y agregar una línea con el puerto que quiero, por ejemplo server.port=8081, y listo, el proyecto levanta por ahí.

Conclusiones:

Indicar 5 conclusiones que llegó después de los temas tratados de manera práctica en este laboratorio.

- **En conclusión, esta primera clase práctica me sirvió bastante para entender cómo estructurar un proyecto básico utilizando Spring Boot. Logré comprobar de forma directa la diferencia entre usar un @Controller para conectar mis datos con una vista web a través del objeto Model, y aplicar un @RestController para devolver la información en formato JSON puro. Armar las clases desde cero en IntelliJ y ver el resultado funcionando en el navegador me dejó una base muy sólida y clara sobre cómo interactúan estos componentes para los próximos laboratorios.**